

FIREWALL SIMULATION

Object-Oriented Programming (OOP) Final Project

Prepared by

Shahad Al-Qurashi

Noran Malibari

Layan Al-Harbi

Supervised by: Mashael Allahaybi

Course Name & Code: Object-Oriented Programming (OOP) &
CS1704

Section No: 143

Group No: 4

Date: May 2026

1. Introduction

Overview:

In today's digital world, network security is a critical priority. This project aims to design and implement a Firewall Traffic Simulator, a Java-based application developed to manage, monitor, and filter network traffic records. The system acts as a first line of defense, analyzing incoming web traffic data to decide whether to allow or block connections based on predefined security rules.

Objectives:

The primary goal of this project is to apply core **Object-Oriented Programming (OOP)** principles to a real-world cybersecurity scenario. By utilizing concepts such as **Inheritance, Encapsulation, and Polymorphism**, the system efficiently handles different types of network data, ensures data integrity through validation, and provides an automated filtering mechanism to detect malicious activities.

Key Features:

- **Automated Security Filtering:** Analyzing URLs and ports to prevent unauthorized access.
- **Data Management:** Storing and searching through traffic records using structured arrays.
- **Duplicate Detection:** Identifying redundant or suspicious repetitive access attempts.
- **User Interaction:** An interactive command-line interface for managing firewall rules in real-time.

2. System Functionalities

The Firewall Traffic Simulator provides a robust set of features to manage, analyze, and store network traffic securely. The core functionalities of the system include:

2.1 System Authentication: Restricts system access to authorized administrators using secure login credentials.

2.2 Data Persistence (File I/O): Automatically loads saved traffic records at startup and allows exporting the current database to FirewallReport.txt for permanent storage.

2.3 Input Validation & Error Handling: Captures detailed network data (IP, Port, URL) while automatically validating IPv4 formats and preventing system crashes from invalid user inputs.

2.4 Automated Security Filtering: Analyzes incoming web traffic against predefined rules, automatically blocking connections targeting malicious URLs or unsafe ports.

2.5 Duplicate Detection (Rate Limiting): Scans the system to detect and flag repeated access attempts from the same IP to the same URL, marking them for monitoring (LOG_ONLY).

2.6 Search Mechanism: Enables administrators to quickly locate specific traffic records using the Source IP address.

2.7 Comprehensive Reporting: Generates formatted, real-time summaries of all stored traffic, displaying both general network details and specific web attributes.

3. Program output example

```
Username: admin
Password: 1234
```

1- Secure login screen requiring admin credentials to access the firewall dashboard.

```
--- Firewall Simulator Menu ---
1. Add New Web Traffic
2. Search Records
3. Display All Stored Records
4. Save Records to Text File
5. Exit
Select an option:
```

2- An easy-to-use command-line interface for managing all firewall operations.

```
Select an option: 1
Enter IP: 192.22.10.18
Enter Port (Available: 80, 443, 8080): 80
Enter Protocol (Choose: TCP, UDP, HTTP): TCP
Enter URL: www.uqu.com
Enter Browser: brave
Success: Traffic record added to system.
```

3- Capturing web traffic details and automatically applying security filters based on predefined rules.

```
Select an option: 2
Enter IP to search: 192.10.56.39
Searching for IP: 192.10.56.39
>>> Web Traffic Analysis <<<
Traffic from: 192.10.56.39 on port: 80 [TCP]
Target URL: www.malicious.com
User-Agent: safari
Security Status: [BLOCK]
```

4- Quickly retrieving specific network records from the system using the source IP address.

```
Select an option: 3
===== ALL NETWORK TRAFFIC REPORT =====
Record #1
>>> Web Traffic Analysis <<<
Traffic from: 192.99.45.33 on port: 80 [TCP]
Target URL: www.uqu.com
User-Agent: brave
Security Status: [ALLOW]
-----
Record #2
>>> Web Traffic Analysis <<<
Traffic from: 192.10.56.39 on port: 80 [TCP]
Target URL: www.malicious.com
User-Agent: safari
Security Status: [BLOCK]
```

5- A detailed summary of all analyzed traffic, showing both network and web-specific details.

```
Select an option: 4
Success: All records saved to FirewallReport.txt
```

6- Successfully exporting and storing all system records into a permanent text file.

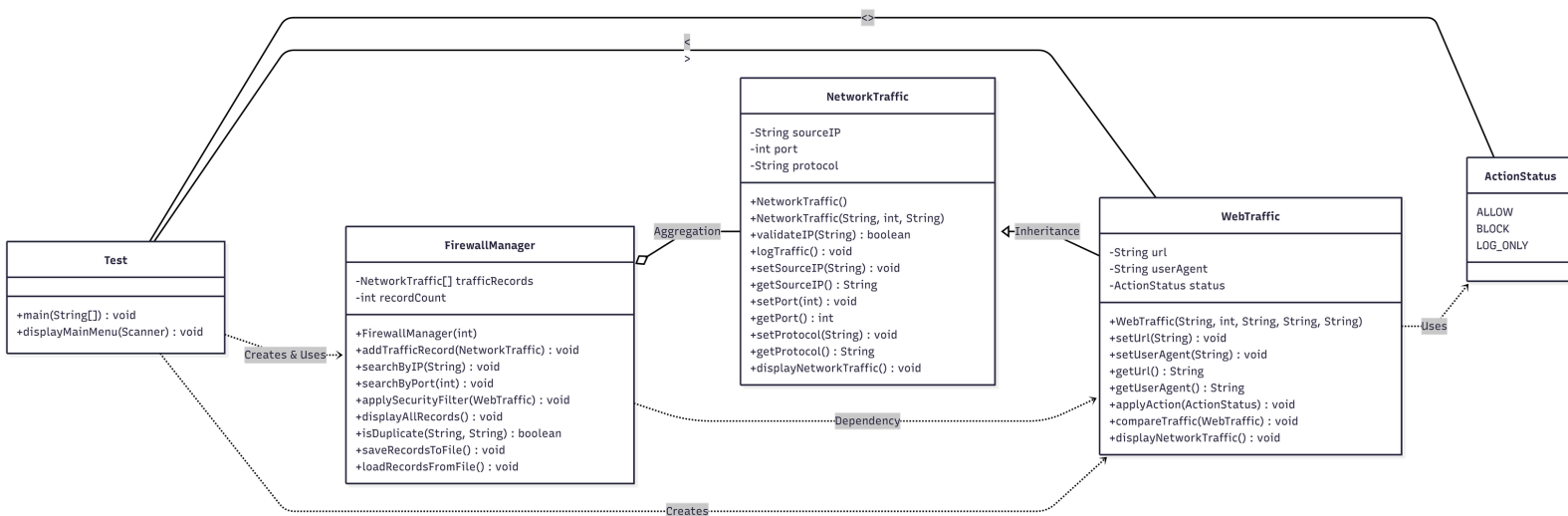
```
Select an option: 5
Closing System...
```

7- Closes the application and stops all processes after ensuring the final system state is reached.

4. System architecture (UML)

UML Class Diagram Overview:

This diagram illustrates the structural architecture of the Firewall Traffic Simulator and highlights the Object-Oriented Programming (OOP) relationships between its classes:



- **Inheritance:** The **WebTraffic** class extends the base **NetworkTraffic** class, allowing it to inherit general network attributes (IP, Port) while introducing web-specific properties (URL, User-Agent).
- **Aggregation:** The **FirewallManager** class manages a collection of records. It contains an array of **NetworkTraffic** objects, representing a "Has-a" relationship for data storage.
- **Dependencies & Enums:** The system utilizes the **ActionStatus** Enumeration to standardize security decisions (ALLOW, BLOCK, LOG_ONLY). Additionally, **FirewallManager** depends directly on **WebTraffic** to apply specific security filters.
- **System Control Flow:** The **Test** class serves as the main entry point of the application. It handles user authentication, displays the interactive menu, and directs the **FirewallManager** to execute core operations.

5. Work distribution and technical contributions

Member name	Responsible	Technical skills and contributions
Layan Al-Harbi	Data Management & Core Architecture	● Developed the NetworkTraffic superclass and FirewallManager class. ● Implemented array-based storage for managing traffic records. ● Created overloaded search methods (searchByIP and searchByPort).
Shahad Al-Qurashi	Security Logic & Advanced OOP	● Designed the WebTraffic subclass demonstrating Inheritance and Encapsulation. ● Implemented the automated security filtering logic and ActionStatus Enum. ● <i>Developed duplicate detection logic utilizing Safe Object Downcasting with type validation</i> ● Applied Method Overriding for polymorphic data display.
Noran Malibari	System Integration & Data Persistence	● Developed the Test main class with system authentication and an interactive menu. ● Implemented File I/O operations for saving and loading records from text files. ● Applied Exception Handling (try-catch) to prevent system crashes from invalid inputs.

6. Conclusion

The Firewall Traffic Simulator is a Java-based application that utilizes core OOP principles to monitor and manage network traffic. It automatically filters malicious URLs, detects duplicate requests, and securely stores data records.